

MLE5004: Computer systems architecture

VI. Assembly basics

Contents

6.1	Source line format	2
6.2	Expressions	3
6.2.1	Operands specification modes	3
6.3	Using operators	4
6.3.1	Bit shifting operators	5
6.3.2	Bitwise operators	5
6.3.3	Type operators	5
6.4	Directives	6
6.4.1	The SEGMENT directive	6
6.4.2	Data definition directives	7
6.4.3	EQU directive	8
6.5	Examples	9
6.5.1	Implicit rules for prefixing an offset with the corresponding segment register .	9
6.5.2	Data definition	9

Definition 6.1 (Machine Language of a Computing System (CS)). the set of the machine instructions to which the processor directly reacts. These are represented as bit strings with predefined semantics.

Definition 6.2 (Assembly Language). a programming language in which the basic instructions set corresponds with the machine operations and which data structures are the machine primary structures. This is a **symbolic language**. *Symbols = Mnemonics + labels.*

The basic elements with which an *assembler* works with are:

- **labels** – user-defined names for pointing to data or memory areas.
- **instructions** - mnemonics which suggests the underlying action. The assembler generates the bytes that codifies the corresponding instruction.
- **directives** - indications given to the assembler for correctly generating the corresponding bytes, e.g. relationships between the object modules, segment definitions, conditional assembling, data definition directives.
- **location counter** – an integer number managed by the assembler for every separate memory segment. At any given moment, the value of the location counter is the number of the generated bytes correspondingly with the instructions and the directives already met in that

segment (the current offset inside that segment). The programmer can use this value (read-only access!) by specifying in the source code the '\$' symbol. Every segment has its own location counter.

NASM supports two special tokens in expressions, allowing calculations to involve the current assembly position: the \$ and \$\$ tokens. \$ evaluates to the assembly position at the beginning of the line containing the expression; so you can code an infinite loop using JMP \$. \$\$ evaluates to the start of the current section; so you can tell how far into the section are by using (\$-\$).

```

1 section .data
2 db 'hello'
3 db 'h', 'e', 'l', 'l', 'o'
4 data_segment_size equ $-$

```

In the above listing:

- \$-\$ is the distance from the beginning of the segment, it is a scalar (constant numerical value).
- \$ - is an offset = **POINTER TYPE**, it is an address.
- \$\$ - is an offset = **POINTER TYPE**. it is an address.
- \$ means "address of here".
- \$\$ means "address of start of current section".
- \$-\$ means "current size of section". For the above example this will be 10, as there are 10 bytes of data given.

6.1 Source line format

In the x86 assembly language the source line format is:

[label[:]] [prefixes] [mnemonic] [operands] [; comment]

Listing 1 presents some examples.

Listing 1: Source line format.

```

1 here: jmp here ; label + mnemonic + operand + comment
2 repz cmpsd ; prefix + mnemonic + comment
3 start: ; label + comment
4 ; just a comment (which could be missed)
5 a dw 19872, 42h ; label + mnemonic + 2 operands + comment
6 len equ $-a ; label + mnemonic + $-a (operand) + comment

```

The allowed characters for a label are:

- Letters: A-Z, a-z;
- Digits: 0-9;
- Characters: _, \$, \$\$, #, @, ~, . and ?

A valid variable name starts with a letter, _ or ?. These rules are valid for all valid identifiers (symbolic names, such as variable names, label names, macros, etc).

All identifiers are *case sensitive*, the language making the distinction between upper and lower case letters while analyzing user defined identifiers. This means that the *Abc* identifier is different from the *abc* identifier. For implicit names which are part of the language (such as keywords,

mnemonics, registers) there are no differences between upper and lower case letters (they are case insensitive).

The assembly language offers two categories of labels:

1. **Code labels**, present at the level of instructions sequences for defining the destinations of the control transfer during a program execution. They can appear also in data segments!
2. **Data labels**, which provide symbolic identification for some memory locations, from a semantic point of view being similar with the variable concept from the other programming languages. They can appear also in code segments!

Observation 6.1. *The value associated with a label in assembly language is an integer number representing the address of the instruction or directive following that label.*

The distinction between accessing a variable's address or its associated content is made as follows:

- When specified in **straight brackets**, the **variable name denotes the value of the variable**; for example, `[p]` specifies accessing the value of the variable, in the same way in which `*p` represents dereferencing a pointer (accessing the content indicated by the pointer) in C;
- In any other context, **the name of the variable represents the address of the variable**; for example, `p` is always the address of the variable `p`;

Listing 2 presents some examples.

Listing 2: Using labels.

```
1  mov EAX, et      ; loads into EAX register the address (offset) of data
2                    ; or code starting at label et
3  mov EAX, [et]    ; loads into EAX register the content from address et (4 bytes)
4  lea eax, [v]     ; loads into EAX register the address (offset) of variable v
5                    ; it loads 4 bytes
```

There are 2 types of *mnemonics*: *instructions* names and *directives* names. *Directives* guide the assembler. They specify the particular way in which the assembler will generate the object code. *Instructions* are actions that guide the processor.

Operands are parameters which define the values to be processed by the instructions or directives. They can be *registers*, *constants*, *labels*, *expressions*, *keywords* or *other symbols*. Their semantics depends on the mnemonic of the associated instruction or directive.

6.2 Expressions

Expression = operands + operators. Operators indicate how to combine the operands for building an expression. **Expressions are evaluated at assembly time** (their values are computable at assembly time, except for the operands representing registers contents, that can be evaluated only at run time – the offset specification formula).

6.2.1 Operands specification modes

Instructions operands may be specified in 3 different ways, called specification modes.

The 3 operand types are: immediate operands, register operands and memory operands. Their values are computed at assembly time for the immediate operands and for the direct addressed operands (the offset part only!), at loading time for memory operands in direct addressing mode (as a complete FAR address – segment address is determinable here so the whole FAR address is

known now !) – this step involves a so called **address relocation** process (adjusting an address by fixing its segment part), and at run time for the registers operands and for indirectly accessed memory operands.

?? : *offset* (assembly time) 0708 : *offset* (loading time)

6.3 Using operators

Operators – used for combining, comparing, modifying and analyzing the operands. Some operators work with integer constants, others with stored integer values and others with both types of operands.

It is very important to understand the difference between operators and instructions. Operators perform computations only with constant scalar values computable **at assembly time** (scalar values = constant immediate values), with the exception of adding and/or subtracting a constant from a pointer (which will issue a pointer data type) and with the exception of the offset computation formula (which supports the ‘+’ operator).

Instructions perform computations with values that may remain unknown (and this is generally the case) until run time. Operators are relatively similar with those in C. *Expression evaluation is done on 64 bits*, the final results being afterwards adjusted accordingly to the sizeof available in the available usage context of that expression.

For example the addition operator (+) performs addition at assembly time and the ADD instruction performs addition during run time. Table 1 gives the operators that are used by the x86 assembly language expressions in NASM.

Table 1: Operators, priority and description.

Priority	Operator	Type	Result
7	-	unary, prefix	Two's complement (negation): $-X = 0 - X$
7	+	unary, prefix	No effect (provides simetry to „-“): $+X = X$
7	~	unary, prefix	One's complement: <code>mov al, ~0 => mov AL, 0xFF</code>
7	!	unary, prefix	Logic negation: $!X = 0$ when $X \neq 0$, else 1
6	*	Binary, infix	Multiplication: $1 * 2 * 3 = 6$
6	/	Binary, infix	Result (quotient) of unsigned division: $24 / 4 / 2 = 3$
6	//	Binary, infix	Result (quotient) of signed division: $-24 // 4 // 2 = -3$ (-24 / 4 / 2 $\neq$ -3!)
6	%	Binary, infix	Remainder of unsigned division: $123 \% 100 \% 5 = 3$
6	%%	Binary, infix	Remainder of signed division: $-123 \% 100 \% 5 = -3$
5	+	Binary, infix	Sum: $1 + 2 = 3$
5	-	Binary, infix	Subtraction: $1 - 2 = -1$
4	<<	Binary, infix	Bitwise left shift: $1 << 4 = 16$
4	>>	Binary, infix	Bitwise right shift: $0xFE >> 4 = 0x0F$
3	&	Binary, infix	AND: $0xF00F \& 0xFF6 = 0x0006$
2	^	Binary, infix	Exclusive OR: $0xFF0F \wedge 0xFF = 0xFF0$
1		Binary, infix	OR: $1 2 = 3$

The indexing operator has a widespread use in specifying indirectly addressed operands from memory.

Consider the following example:

$$5|6 + 7\&8 = (5|6) + (7\&8) = 7 + 0 = 7?$$

The above answer is **wrong** because of the operators precedence, if we take into account the precedence of operators we have:

$$5|6 + 7\&8 = 5|(6 + 7)\&8 = 5|13\&8 = 5|8 = 13 = 0Dh$$

Table 2: Bitwise operators.

Operator	Syntax	Meaning
<code>~</code>	<code>~expr</code>	Bits complement
<code>&</code>	<code>expr1 & expr2</code>	Bitwise AND
<code> </code>	<code>expr1 expr2</code>	Bitwise OR
<code>^</code>	<code>expr1 ^ expr2</code>	Bitwise XOR

6.3.1 Bit shifting operators

expression `>>` *how_many* and *expression* `<<` *how_many*

As an example, consider the listing 3.

Listing 3: Bit shifting.

```

1  mov AX, 01110111b << 3 ; AX = 00000011 10111000b
2  add BH, 01110111b >> 3 ; the source operator is 00001110b

```

6.3.2 Bitwise operators

Bitwise operators perform bit-level logical operations for the operand(s) of an expression. The resulting expressions have constant values. Table 2 presents these operators.

Examples (assume that the expression is represented on a byte):

```

1  ~11110000b ; output result is 00001111b
2  01010101b & 11110000b ; output result is 01010000b
3  01010101b | 11110000b ; output result is 11110101b
4  01010101b ^ 11110000b ; output result is 10100101b

```

6.3.3 Type operators

They specify the types of some expressions or operands stored in memory. Their syntax is:

type expression

where the type specifier is one of the following: **BYTE**, **WORD**, **DWORD**, **QWORD** or **TWORD**.

This syntax causes **expression** to be treated temporarily (limited to that particular instruction) as having „**type**” sizeof without destructively modifying its initial value. That is why these type operators are also called „non-destructive temporary conversion operators”. For memory stored operators, type may be **BYTE**, **WORD**, **DWORD**, **QWORD** or **TWORD** having the size of 1, 2, 4, 8, and 10 bytes respectively. For code labels type is either NEAR (4 bytes address) or FAR (6 bytes address).

For example, **byte** [A] takes only the first byte from the memory location designated by A. Similar, **dword** [A] will consider the double word starting at address A.

BYTE / WORD / DWORD / QWORD specifiers have always the task of clarifying an ambiguity. This is true also when we talk about a memory variable, writing `mov BYTE [v], 0` or `mov WORD [v], 0` is also a way of eliminating an ambiguity, because NASM does not associate a DATA TYPE to a variable! v is NEVER a byte/ word/ dword, it is just a label without an associated data type.

The QWORD specifier is **never** explicitly present in any 32 bits code.

```
1  mov [v],0 ; syntax error - operation size not specified
```

Listing 4 gives some examples where we need a data size specifier (type operator) for the given operands:

Listing 4: Type operators, examples where type operators are needed.

```
1  mov [mem], 12
2  (i)div [mem]
3  (i)mul [mem]
4  push [mem]
5  pop [mem]
6  push 15      ; this is an inconsistency of NASM, the assembler will not issue
7               ; an error or warning, but instead it will generate: push DWORD 15
```

6.4 Directives

Directives direct the way in which code and data are generated during assembling.

6.4.1 The SEGMENT directive

The SEGMENT directive allows targeting the bytes of code or of data emitted by an assembler to a given segment, having a name and some specific characteristics. The syntax is:

SEGMENT name [type][ALIGN = alignment][combination][usage][CLASS = class]

The numeric value assigned to the segment name is the segment address (32 bits) corresponding to the memory segment's position during run-time. For this purpose, NASM offers the special symbol \$\$ which is equal with the current segment's address, this having the advantage that can be used in any context, without knowing the current segment's name.

Except the name, all the other fields are optional both regarding their presence or the order in which they are specified.

The optional arguments alignment, combination, usage and 'class' give to the link-editor and the assembler the necessary information regarding the way in which segments must be loaded and combined in memory.

The type allows selecting the usage mode of the segment, having the following possible values:

- **code** (or **text**) - the segment will contain code, meaning that the content cannot be written but it can be executed
- **data** (or **bss**) - data segment allowing reading and writing but not execution (implicit value).
- **rdata** - the segment that it can only be read, containing definitions of constant data

The optional argument alignment specifies the multiple of the bytes number from which that segment may start. The accepted alignments are powers of 2, between 1 and 4096.

If alignment is missing, then it is considered implicitly that ALIGN=1, i.e. the segment can start from any address.

The optional argument combination controls the way in which similar named segments from other modules will be combined with the current segment at linking time. The possible values are:

- **PUBLIC** - indicates to the link editor to concatenate this segment with other segments with the same name, obtaining a single segment having the length the sum of concatenated segments' lengths.
- **COMMON** - specifies that the beginning of this segment must overlap with the beginning of all segments with the same name, obtaining a segment having the length equal to the length of the larger segment with the same name.
- **PRIVATE** - indicates to the link editor that this segment cannot be combined with others with the same name.
- **STACK** - the segments with the same name will be concatenated. During run time the resulted segment will be the stack segment.

Implicitly, if no combination method is specified, any segment is **PUBLIC**.

The argument *usage* allows choosing another word size than the default 16 bits one.

The argument 'class' has the task to allow choosing the order in which the link editor puts the segments in memory. All the segments that have the same class will be placed in a contiguous block of memory whatever their order in the source code. No implicit value exists, it being undefined when its specification is missing, leading though to NOT concatenating all the program's segments defined so in a continuous memory block.

Examples:

segment code use32 class = CODE

segment data use32 class = DATA

6.4.2 Data definition directives

Data definition = declaration (attributes specification) + allocation (reserving required mem. space).

The data definition part is *unique*, the declaration part is *not unique*, the allocation part is *unique*.

The general form of a data definition source line is:

[name] data_type expression_list [; comment]

or

[name] allocation_type factor [; comment]

or

[name] TIMES factor data_type expression_list [; comment]

where *name* is a label for data referral. The *data type* is the size of representation and its value will be the address of its first byte. *factor* is a number which shows how many times the *expression_list* is repeated.

Data_type is a data definition directive, one of the following:

- **DB** - byte data type (BYTE)
- **DW** - word data type (WORD)
- **DD** - doubleword data type (pointer - DWORD)
- **DQ** - 8 bytes data type (QWORD – 64 bits)
- **DT** - 10 bytes data type (TWORD – used to store BCD constants or real constants for extended precision)

For example, the code from Listing 5 following sequence defines and initializes 5 memory variables.

var1 and *var2* variables are defined using common labels, visible in the entire source code, while *.a* and *.b* are local labels, the access to these local variables being restricted in the sense that:

Listing 5: Define and initialize five memory variables.

```

1 segment data
2     var1    DB 'd'    ;1 byte
3     .a      DW 101b    ;2 bytes
4     var2    DD 2bfh    ;4 bytes
5     .a      DQ 3070    ;8 bytes (1 quadword)
6     .b      DT 100     ;10 bytes

```

- these variables can be accessed with the local name, i.e. `.a` or `.b`, until another common label is defined (they are local to preceding label);
- they can be accessed from anywhere by their complete name: `var1.a`, `var2.a` or `var2.b`.

The initialization value may be also an expression, as for example `vartest DW (1002/4+1)`.

After a data definition directive may appear more than one value, thereby allowing declaration and initialization of arrays. For example, the declaration:

Tablou DW 1, 2, 3, 4, 5

creates an array with 5 integers represented as words and having their values 1,2,3,4,5. If the values supplied after the directive don't fit on a single line, we can add as many lines as necessary, which shall contain only the directive and the desired values. Example:

Tabsqrs DD 0, 1, 4, 9, 16, 25, 36
DD 49, 64, 81
DD 100, 121, 144, 169

`allocation_type` is a uninitialized data reservation directive:

- **RESB** - byte data type (BYTE)
- **RESW** - word data type (WORD)
- **RESD** - doubleword data type (DWORD)
- **RESQ** - 8 bytes data type (QWORD - 64 bits)
- **REST** - 10 bytes data type (TWORD - 80 bits)

`factor` is a number showing how many times the specified data type allocation is repeated. For example, `Tabzero RESW 100h` reserves 256 words for `Tabzero` array.

The **TIMES** directive allows repeated assembly of an instruction or data definition.

TIMES factor data_type expression

For example, `Tabchar TIMES 80 DB 'a'` - creates an "array" of 80 bytes, every one of them being initialized with the ASCII code of 'a'.

TIMES can also be applied to instructions: `TIMES 32 add eax, edx` - has as effect $EAX = EAX + 32 * EDX$.

6.4.3 EQU directive

The **EQU** directive allows assigning a numeric value or a string during assembly time to a label without allocating any memory space or bytes generation. The **EQU** directive syntax is:

name EQU expression

Listing 6 gives some examples.

Listing 6: EQU directive - examples of use.

```

1  END_OF_DATA EQU '!'
2  BUFFER_SIZE EQU 1000h
3  INDEX_START EQU (1000/4 + 2)
4  VAR_CICLARE EQU i
5  ; - the expressions used when defining labels by EQU may
6  ; also contain labels defined by EQU:
7  TABLE_OFFSET EQU 1000h
8  INDEX_START EQU (TABLE_OFFSET + 2)
9  DICTIONAR_STAR EQU (TABLE_OFFSET + 100h)

```

6.5 Examples

6.5.1 Implicit rules for prefixing an offset with the corresponding segment register

Listing 7 gives some examples of implicit rules used for prefixing an offset with an appropriate segment register.

Listing 7: Implicit prefixing of an offset with a segment register.

```

1  mov EAX, [v]           ; mov EAX, DWORD PTR DS:[405000]
2  mov EAX, [EBX]         ; mov EAX, DWORD PTR DS:[EBX]
3  mov EAX, [EBP]         ; mov EAX, DWORD PTR SS:[EBP]
4  mov EAX, [EBP*2]       ; mov EAX, DWORD PTR SS:[EBP+EBP]
5  mov EAX, [EBP*3]       ; mov EAX, DWORD PTR SS:[EBP+EBP*2]
6  mov EAX, [EBP*4]       ; mov EAX, DWORD PTR DS:[EBP*4]
7  mov EAX, [EBX+ESP]     ; ESP - base... EBX - index ; EAX <- dword ptr [SS:ESP+EBX]...
8  mov EAX, [ESP + EBX]   ; ESP - base... EBX - index ; EAX <- ...SS:...
9  mov EAX, [EBX+ESP*2]   ; syntax error BECAUSE ESP can be ONLY a base register!
10 mov EAX, [EBX+EBP*2]   ; mov EAX, DWORD PTR [DS:EBX+EBP*2]
11 mov EAX, [EBX+EBP]     ; ...DS...
12 mov EAX, [EBP+EBX]     ; ...SS...
13 mov EAX, [EBX*2+EBP]   ; ...SS...
14 mov EAX, [EBX*1+EBP]   ; ...SS...
15 mov EAX, [EBP*1+EBX]   ; ...DS...
16 mov EAX, [EBX*1+EBP*1] ; ...SS... - the first found scaled element is taken
17                       ; as index !! EBP - base
18 mov EAX, [EBP*1+EBX*1] ; ...DS... - the first found scaled element is taken
19                       ; as index !! EBX - base
20 mov EAX, [EBP*1+EBX*2] ; ...SS...

```

6.5.2 Data definition

Listing 8: Data definition.

```

1 segment data
2 a1 db 0,1,2,'xyz' ; 00 01 02 'x' 'y' 'z'
3 ; offset(a1) - computed at loading time = 00401000 (OllyDbg)
4 ; offset(a1) determined at assembly time by NASM = 0 !
5 db 300, "F"+3 ; 2C, ASCII code for 'F'+3 = 49
6 ; Warning - byte data (300=1 2Ch) exceeds bounds!
7 a2 TIMES 3 db 44h ; 44 44 44
8 ; offset(a2) = 00401008,
9 ; assembly time offset = 8!
10 a3 TIMES 11 db 5,1,3 ; 05 01 03...11 times (total 33 bytes)
11 a41 db a2+1 ; syntax err. - OBJ format can only handle 16 or 32
12 ; bits relocation !
13 a4 dw a2+1, 'bc' ; offset(a2)=00401008h; a2+1=00401009h; 09 10 'b' 'c' =
14 ; 09 10 62 63 (2 words) ; 09 10 (correct, BUT... this
15 ; particular 10h value it is only finally computable after
16 ; LOADING!) - so the offset of the beginning of segments is
17 ; computable only at LOADING TIME!)
18 ; The variables offset relative to the beginning of segments
19 ; in which they are defined are constant POINTERS
20 ; (so POINTER data types!, NOT scalar!) determinable at
21 ; assembly time !
22 a42 dw a2+1, 'b', 'c' ; 09 10 'b' 00 'c' 00 = 09 10 62 00 63 00 (3 words!)
23 a44 dw 1009h ; 09 10
24 a5 dd a2+1, 'bcd' ; 09 10 40 00| 62 63 64 00
25
26 a6 TIMES 4 db '78' ; 37 38 37 38 37 38 37 38
27 a61 TIMES 4 db '7', '8' ; 37 38 37 38 37 38 37 38
28 a62 TIMES 4 dw '78' ; 37 38 37 38 37 38 37 38
29
30 a7 db a2 ; syntax err. OBJ format can only handle 16 or 32 bits relocation
31 ; (equiv. mov ah, a2)
32 a8 dw a2 ; 08 10
33 a9 dd a2 ; 08 10 40 00
34
35 a11 db [a2] ; syntax error!
36 a12 dw [a2] ; syntax error !
37 a13 dd dword [a2] ; syntax error !
38 a14 dq [a2] ; syntax error !
39 a15 dd eax ; expression syntax error
40 a16 dd [eax] ; expression syntax error
41 mov ax, v ; Warning - 32 bit offset in 16 bit field!

```

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press.

<https://doi.org/10.1201/b14582>

3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.